ETHEKWINI EMERGENCY SERVICES

Criticom V5 Solution Wiki

The complete reference for the emergency dispatch, response and command platform — architecture, every module, the data model, the responder app, automation, security and operations.

Version
Wave 5 · Web 110 routes · Mobile 1.1.0

Issued
4 August 2026

Status
Production

Audience
Engineering · Operations · Command

Contents

I What Criticom Is

1.1 The problem it solves · 1.2 The five departments · 1.3 Lineage from V4 · 1.4 The system in numbers

II Architecture

2.1 Topology · 2.2 Technology choices and why · 2.3 Repository layout · 2.4 The life of a call

III Authorization

3.1 Silos · 3.2 The roles · 3.3 The route matrix · 3.4 Row-level security · 3.5 Privileged functions

IV The Console — Module by Module

4.1 Dashboard · 4.2 Call intake · 4.3 Dispatch · 4.4 Incidents · 4.5 Responder
4.6 Metro Police · 4.7 Fire & Rescue · 4.8 EMS · 4.9 Disaster Management
4.10 Case management · 4.11 HR & leave · 4.12 Roster & duty · 4.13 Performance
4.14 Reporting · 4.15 CCTV · 4.16 Administration · 4.17 Criti · 4.18 Search

V The Responder App

5.1 Screens · 5.2 The duty gate · 5.3 The offline outbox · 5.4 Officer-down · 5.5 Alerting and push

VI The Automation Engine

6.1 Why it exists · 6.2 The seven rules · 6.3 Anti-spam · 6.4 Scheduling and failure

VII Data Model

7.1 Domains · 7.2 The incident record · 7.3 Stored procedures · 7.4 Migrations

VIII Integrations

8.1 Catalogue · 8.2 The proxy pattern · 8.3 CCTV cut-over

IX Security

9.1 Model · 9.2 Headers and transport · 9.3 Rate limiting · 9.4 Audit · 9.5 Outstanding items

X Operations

10.1 Environments · 10.2 Deploying · 10.3 Backup and recovery · 10.4 First response

XI Engineering Standards

11.1 CI · 11.2 Testing · 11.3 Linting · 11.4 Conventions

XII Appendices

A Route index · B Table index · C Migrations · D Environment variables · E Glossary

PART I

What Criticom Is

> Criticom is the command-and-control platform for eThekwini's emergency services. One call-taker logs an incident; the platform routes it to the right departments, puts it on a map, pushes it to the crews' phones, records everything they do at the scene, and keeps the evidential trail that an inquiry will read a year later.

1.1 The problem it solves

Five emergency services operate across eThekwini — Metro Police, Fire & Rescue, EMS, Disaster Management, and the central Control Room that fields the calls. They share a city, they share incidents, and they routinely need each other: a structure fire needs Fire and EMS and often Police for the cordon.

What they did not share was a system. Each department kept its own records, so a call that touched three services produced three partial accounts of it, and the only thing joining them was the radio. Criticom's premise is that the incident is the record, and every department writes to the same one.

Three properties follow from that, and they shape almost every design decision in this document:

- **Departments must see their own work and no more.** A paramedic has no business reading a Metro Police case file. But when a call is routed to both, both must see it. That tension is resolved once, in the database, by row-level security — not per screen.
- **The record is evidence.** Timestamps, who did what, which crew attended, what the scene looked like. These get read in disciplinary hearings, insurance disputes and inquests. A timestamp that records "when the network came back" rather than "when the crew arrived" is worse than no timestamp, because it looks authoritative.
- **Nothing may depend on good conditions.** Coverage fails in basements and industrial areas. Phones lock. Screens get ignored during a major incident. The platform is built so that each of those degrades rather than loses data.

1.2 The five departments

Departments are called **silos** throughout the codebase and the database. The term is deliberate: it names the isolation boundary, which is a security concept, not an organisational chart.

SILO	SERVICE	SCOPE
`Ethekwini`	Control Room	Global. Takes calls, routes them, sees every department's incidents. The co-ordinating tier.
`MetroPolice`	Metro Police	Law enforcement, by-law, traffic, firearms register, SIRA security-guard reports.
`MetroFire`	Fire & Rescue	Fire suppression, rescue, hydrant register, post-incident review.
`Medical`	EMS	Ambulance response, patient care records, hospital handover, restocking.
`EthekwiniDisaster`	Disaster Management	Floods, major incidents, mutual-aid partners, relief stock, post-mortem reporting.

Each carries its own identity — official name, short name, monogram, colours, motto — stored in `department_branding` and editable from Admin. Crests are *uploaded*, never shipped with the app: official municipal and service insignia belong to the municipality, so the platform does not fabricate them. Until one is uploaded, the interface renders a monogram badge in that department's colours, so nothing looks unfinished.

1.3 Lineage from V4

Criticom V5 is a rebuild of Criticom V4, an ASP.NET MVC application of roughly 38 controllers backed by five separate SQL Server databases. V5 reproduces every V4 capability and consolidates the five databases into one Postgres schema with row-level security doing the separation the separate databases used to do badly.

The migration ran in five waves. Waves 1–3 reached functional parity; wave 4 aligned V5 with a July feature drop in V4; wave 5 — the subject of much of this document — added what a production emergency service needs that V4 never had: offline field operation, an officer-down path, automatic escalation, cross-module search, rate limiting and a test suite.

1.4 The system in numbers

110 WEB ROUTES	**23** API ENDPOINTS	**57** TABLES	**41** STORED PROCEDURES
25 ROLES	**28** MIGRATIONS	**9** MOBILE SCREENS	**47** TESTS

Roughly 36,000 lines of TypeScript across the console, the responder app and two shared packages, plus 5,100 lines of SQL. The SQL number matters more than it looks: a substantial part of this system's behaviour

— dispatch, escalation, search, rate limiting, authorization — lives in the database rather than in application code, for reasons set out in Part II.

PART II

Architecture

2.1 Topology

COMPONENT	RUNS ON	RESPONSIBILITY
Console (web)	Vercel · Next.js 15, React 19	Every operator-facing screen. Server-rendered per request; no page is statically cached, because every page is user-scoped.
Responder app	Expo SDK 52 / React Native 0.76	The crew's phone. Distributed through EAS; JavaScript changes ship over the air.
Database	Supabase Postgres 15 + PostGIS	Data, authorization, realtime, storage, and a meaningful share of the business logic.
Automations	`pg_cron`, inside the database	Escalation, expiry warnings, digests. No separate worker to deploy or monitor.

2.2 Technology choices and why

Logic in the database, not only in the app

Dispatching a call writes an incident row, updates unit state, appends a timeline entry, writes an audit record and raises one alert per receiving department. As application code that is six round trips that can half-succeed — and a half-dispatched call is a crew that never rolls.

So those workflows are stored procedures. `route_incident()`, `dispatch_unit()`, `raise_panic()`, `update_responder_status()` each do the whole job in one transaction. Web and mobile call the same procedure, so the two surfaces cannot drift.

> **THE TRADE-OFF**
> Logic in SQL is harder to test and harder for a TypeScript developer to change. It is accepted here only where atomicity is the point. Reporting, KPI arithmetic and formatting stay in TypeScript (`packages/api`), where they are readable and unit-tested.

Authorization in the database, never only in the app

Row-level security decides what each user can read and write. The application never adds a `WHERE silo = …` for security — only ever to reduce traffic. This is the single most important architectural decision in the system, and it is what makes a shared incidents table safe for five services.

> **CONSEQUENCE**
> A bug in a page cannot leak another department's data, because the page never had it. The database returned the rows that user was entitled to and nothing else.

Realtime over polling

Dispatch boards, incident timelines and the alert bell subscribe to Postgres change streams. RLS applies to the socket as well, so a subscriber is only ever delivered rows they could have queried.

Server-rendered, per request

Every console page is `force-dynamic`. Caching a page that renders one operator's permitted view and serving it to another is precisely the failure this system cannot have.

2.3 Repository layout

A pnpm workspace driven by Turborepo.

PATH	CONTENTS
`apps/web`	The console. Next.js App Router; route groups `(app)` for authenticated screens and `(auth)` for login.
`apps/mobile`	The responder app. Expo Router; `lib/` holds the providers (auth, duty, sync, notifications, branding).
`packages/db`	Supabase client factories, generated database types, Zod schemas, and `roles.ts` — the route matrix.
`packages/api`	Every query and mutation, grouped by module. Both apps import from here; neither builds queries inline.
`supabase/migrations`	28 numbered, append-only migrations.
`scripts`	Migration runner, seed, admin creation, SAPS crime-statistics loader.
`docs`	Reconnaissance, architecture, phases, security, parity, standards, runbook.

2.4 The life of a call

End to end, with the mechanism named at each step.

#	STEP	WHAT ACTUALLY HAPPENS
1	**Call arrives**	A call-taker opens `/dispatch/log-call`. Address autocomplete resolves through the server-side geocoding proxy; the map pin and the coordinates stay in step.
2	**Duplicate check**	`check_duplicate_incident()` looks back 12 hours by caller number, and by location plus type. Two callers reporting one crash produce one incident.
3	**Incident created**	`log_incoming_call()` writes the row, stamps the call-taker, allocates a sequential number from `next_incident_number()`, and audits it.
4	**Routed**	`route_incident()` sets the receiving departments, appends a timeline entry, audits, and raises one alert per department — in a single transaction.
5	**Crews notified**	Realtime delivers to every open screen. High and Critical raise a blocking modal. Expo push covers locked phones.
6	**Unit assigned**	`dispatch_unit()` commits a specific appliance and moves it out of the available pool.
7	**Crew responds**	The responder advances En Route → Arrived → On Scene → Transporting → Resolved. Each step queues locally first and carries the time it was pressed.
8	**Scene recorded**	Photos and voice notes are retained on the device and uploaded to the `incident-media` bucket, filed against the incident.
9	**Report filed**	The attending department completes its own report — police case, fire report, or patient care record.
10	**Closed**	`close_case()` records the reason and audits it. The case-report PDF embeds the timeline, attachments and audit trail.
11	**Watched throughout**	If any step stalls — unassigned too long, no arrival recorded, no update for hours — the automation engine escalates without anybody asking.

PART III

Authorization

> Two independent layers. The **route matrix** decides which screens a role may open. **Row-level security** decides which rows they may touch. The first is convenience and clarity; the second is the actual boundary. Neither is trusted to do the other's job.

3.1 Silos

Every user carries a silo and a role in their JWT app metadata. Three database helpers read them: `current_silo()`, `app_role()` and `is_global()`. Policies are written in terms of those, so a policy reads as the sentence a person would say: *your department's rows, plus anything dispatched to your department, plus everything if you are global.*

> **A DISTINCTION WORTH KEEPING**
>
> `is_global()` answers "may this role see across departments" and is correct for `SELECT`. It returns true for read-only roles such as `Viewer`. Using it to gate writes would grant those roles write access, so a separate `can_write_global()` exists for write policies. Migration 0019 fixed five policies that had conflated the two.

3.2 The roles

The `user_role` enum carries 25 values. **23 are assignable to people** from Admin → Users. The two that are not: `System` is an internal service identity used by automated writes, and `Citizen` exists only for the public location-share flow, where a member of the public posts their GPS against a single-use token and never holds an account at all.

Roles are grouped in `packages/db/src/roles.ts`. Groups compose, so a rule is written once.

GROUP	MEMBERS
`ADMIN_ROLES`	SystemAdmin, SuperAdmin, Admin
`SUPERVISORY_ROLES`	Admin tier + Supervisor, Captain, Chief, FireChief, System
`CALL_TAKER_ROLES`	Admin tier + Operator, ControlRoom, IncidentLogger, System
`DISPATCH_DESK_ROLES`	Admin tier + Dispatcher, Operator, ControlRoom, Supervisor, System
`RESPONDER_ROLES`	Responder, Police, Officer, Metro, Security, Captain, Firefighter, FireChief, Chief, Paramedic, Doctor, Disaster
`POLICE_ROLES`	Captain, Police, Officer, Metro, Security
`FIRE_ROLES`	Chief, FireChief, Firefighter
`MEDICAL_ROLES`	Paramedic, Doctor
`CASE_MANAGER_ROLES`	Admin tier + CaseManager, Dispatcher, Operator, ControlRoom, IncidentLogger, Supervisor, System, Viewer

The two-desk split

`CALL_TAKER_ROLES` and `DISPATCH_DESK_ROLES` deliberately barely overlap. An `IncidentLogger` may capture a call and may not work the dispatch queue; a `Dispatcher` may work the queue and may not capture calls. This mirrors how the control room is actually staffed, and it is pinned by tests — a regression that merged the two desks would be an authorization change disguised as a convenience.

3.3 The route matrix

`ROUTE_RULES` is an ordered list of path prefixes with the roles (and optionally the silo) permitted to open them. `canAccess(path, role, silo)` evaluates it. One list drives:

- The Next.js middleware, which redirects an unauthorized request to `/403`.
- The sidebar and top navigation, which render only reachable destinations.
- The command palette, which only offers pages the user may open.
- Dashboard quick actions.

Because all four read the same list, navigation and enforcement cannot drift — the classic failure where a link exists to a page that then refuses to load.

```
// Ordered: the first matching prefix wins, so specific rules precede general ones.
{ prefix: '/dispatch/log-call', roles: CALL_TAKER_ROLES },
{ prefix: '/dispatch',          roles: DISPATCH_DESK_ROLES },
{ prefix: '/police',           roles: [...POLICE_ROLES, ...ADMIN_ROLES], silo: 'MetroPolice' },
{ prefix: '/roster/mine',      roles: undefined },   // every role
{ prefix: '/search' },                               // every role; RLS scopes results
```

3.4 Row-level security

The standard read policy, and the shape most tables follow:

```
create policy incidents_read on public.incidents for select using (
  silo = public.current_silo()                -- my department owns it
  or public.is_global()                       -- I am control room / admin
  or public.current_silo() = any(dispatched_silos)  -- it was routed to us
);
```

That third clause is what makes multi-department response work. A paramedic sees a Metro Police incident precisely when it was routed to EMS, and not otherwise. This was verified against live data during wave 5: searching as a paramedic returned the one call routed to Medical and not the police-only call beside it.

3.5 Privileged functions

Stored procedures that must bypass RLS run as `SECURITY DEFINER`. Every one of them opens with a role check:

```
perform public.require_role('SystemAdmin','SuperAdmin','Admin','Dispatcher',
                            'Operator','ControlRoom','Supervisor','System');
```

> **HISTORIC DEFECT, NOW CLOSED**
> Before migration 0019, every mutating procedure was `SECURITY DEFINER` with no role check, and `EXECUTE` was granted to `anon`. A caller holding only the public anonymous key could route, dispatch or close any incident in any department — and the audit log would record it as legitimate with a null actor. 0019 added the guards; 0020 fixed the revocations, which had silently been no-ops.

The deliberate exception

`raise_panic()` has no role gate. Any authenticated account may call for help. Gating an emergency on a role matrix is how somebody dies over a provisioning mistake, and the reasoning is written into the migration so a future reviewer does not "fix" it.

Security invoker where possible

`search_everything()` reads across six tables and every department. It is `SECURITY INVOKER`, so RLS runs for the calling user on each branch. Making it `DEFINER` for speed would turn one search box into a cross-department data leak.

PART IV

The Console — Module by Module

> Every operator-facing screen, what it is for, and the decisions behind it. Routes are given as paths; all of them sit under the authenticated `(app)` group unless noted.

4.1 Dashboard /DASHBOARD

Role-aware. A dispatcher gets the live operations board — queue depth, active deployments, a map, an incident feed, SLA gauges. A responder gets their assigned calls and shift status. A supervisor gets headcount and approvals. An administrator gets system health.

The shell above it carries the ops command bar on every page: an external-resources tier (E-NATIS, CAS, PERSAL, FIREARM, FINES, TRACE, FORENSICS, RealPTT, FlightRadar, MarineTraffic) driven by `OpsLink` configuration records, plus an internal module tier filtered by the route matrix. Administrators repoint or add systems without a deployment; placeholders render dimmed until an endpoint is issued.

4.2 Call intake /DISPATCH/LOG-CALL

The primary call-taking screen. Caller details, description, classification driven by the configurable incident-category tree, location by address autocomplete or map pin, and the receiving departments.

- **Duplicate detection** runs before submission — 12-hour window, by caller number and by location plus type.
- **Attribution** stamps the call-taker, which is what the operational KPIs are computed from.
- **Geocoding** goes through a server-side proxy (Photon primary, Nominatim fallback). Cloud egress ranges are frequently blocked by Nominatim, which is why the primary is Photon.
- **Categories** come from `command_records`, so a service adds an incident type from Admin rather than by deployment.

4.3 Dispatch

ROUTE	PURPOSE
`/dispatch/pending`	The queue, severity then call time, live. One status bar carries queue depth as a figure a dispatcher can glance at continuously.
`/dispatch/assign/[id]`	Full-detail assignment console: incident, map, available units, road-routed ETA.
`/dispatch/kpi`	Dispatcher throughput and lifecycle timings.
`/dispatch/ptt`	RealPTT console — text to radio, plus a two-hour audio log.

4.4 Incidents

ROUTE	PURPOSE
`/incidents`	Filterable register of everything the user's department can see.
`/incidents/[id]`	The case file: detail, timeline, attachments, status, citizen GPS request, PDF export.
`/incidents/map`	Live GIS view, severity-coloured markers, clickable pins.

4.5 Responder `/RESPONDER` · `/SHIFT/*`

The console counterpart to the phone. Start of shift is a three-step gate — mental readiness declaration, vehicle selection, vehicle inspection — and it must be completed before a crew may take incidents.

> **WHY THE GATE EXISTS**
> The mental check is a fitness-for-duty declaration (substance use, emotional regulation, reaction time) and the vehicle inspection is a roadworthiness record. Letting somebody answer a call before either exists means there is no record that they were fit to attend — which is exactly the record that matters after an incident goes wrong.

4.6 Metro Police `/POLICE/*`

- **Reports** — seven case-report wizards covering the V4 offence types.
- **Firearms** — the firearm authorization register.
- **Security** — SIRA guard reports with a resolution flow.
- **Crime hotspots** — the SAPS crime-statistics dataset, ranked by station and category.

4.7 Fire & Rescue `/FIRE/*`

- **Reports** — fire incident reports with dispatch, arrival and clear timings, and a PDF export.
- **Hydrants** — the hydrant register on a map, which is operational data during a fire.
- **PIR** — post-incident review, with publication to the public incident feed.

4.8 EMS `/MEDICAL/*`

The full patient care record: history, assessment findings, vital signs, interventions, legal handover to a receiving facility, and signature capture for responder, hospital representative and doctor. `/medical/restock` tracks ambulance consumables.

4.9 Disaster Management /DISASTER/*

Monitoring with a live map and a wind widget for fire-spread modelling; a dispatch view; the mutual-aid partner register (41 partners, seeded verbatim from V4); relief stock; and post-mortem reporting.

4.10 Case management /CASE-MANAGER

Review and closure of resolved incidents, an audit view, and the official case-report PDF — which downloads and embeds the scene photographs from storage rather than merely linking them, because a case report has to be complete as a single document.

4.11 HR and leave V5 ADDITION

ROUTE	PURPOSE
`/hr`	Command centre: headcount, on duty, on leave today, approvals waiting with inline decision, certifications expiring within 30 days.
`/leave`	Balances per member, a month calendar of who is away, reasons, and an approval trail.
`/profile/my-hr`	Self-service: own record, balances, certifications, training.

Every pending request shows its **coverage impact** — the percentage of the department away if granted — because that is the number the approver actually needs and previously had to work out in their head. Entitlements live on `LeaveType` configuration rows, so HR tunes them from Admin.

4.12 Roster and duty

ROUTE	PURPOSE
`/roster`	Shift generators (224 and 24-48 rotations), shifts, assignments, endorsements.
`/roster/allocation`	Post-first duty board: the establishment a shift must fill, with coverage gaps visible, plus a person view.
`/roster/duty-sheet`	The printable DCP duty sheet — shift blocks, on-leave list, special events, sign-off columns.
`/roster/mine`	Open to every role: current duty, next duty, upcoming allocations, leave, endorsements.

Allocations are **published** and then **acknowledged**. Publication makes a draft visible to the member; acknowledgement is the accountability half, and an unacknowledged duty starting soon is one of the things the automation engine chases.

4.13 Performance `/PERFORMANCE/*`

- **Operational KPIs** — call-taker and dispatcher throughput, category pivot, call detail, logons, CSV export. A faithful port of V4's `OperationalKpiService`.
- **My performance** — self-service: own throughput against the team, monthly trend, appraisal snapshot.
- **Plans** — annual 1-July appraisal cycle with mid-year and final scoring and an approval workflow.
- **Rollup** — the same measures aggregated at every level of the reporting hierarchy.
- **Reports** — one date window cut three ways: by member, by department, by incident type.

4.14 Reporting

ROUTE	PURPOSE
`/reports`	Archive of departmental reports.
`/report-hub`	Advanced filtered incident reporting with CSV export and saved templates.
`/reports/builder`	Registry-driven custom report builder: pick a dataset, filter, group, chart.

The builder is the one place users compose queries, so its filter layer is defensive and unit tested: a filter naming a column that is not in the registry is dropped, an operator the field type does not support is dropped, an enum filter may only use values the enum has, and sensitive datasets are closed to ordinary roles.

4.15 CCTV `/CAMERAS`

The room runs on the SANRAL freeway matrix today and moves to Hikvision HikCentral next. Both are in the integration catalogue, and `/api/cctv/cameras` serves whichever is enabled — the cut-over is a toggle, not a deployment. `/api/cctv/stream` brokers signed live-preview and playback URLs; signing is server-side only and every grant is audit-logged for chain of custody. `/cameras/footage` is the retrieval console, capped at a 24-hour window.

4.16 Administration `/ADMIN/*`

ROUTE	PURPOSE
`/admin`	Users and roles. All 24 roles assignable.
`/admin/config`	Incident categories, leave types, shift types, report templates, ops links. Soft delete.
`/admin/personnel`	Personnel register.
`/admin/operators`	Control-room operators and call signs.
`/admin/vehicles`	Fleet.
`/admin/devices`	MDTs, radios, tablets.
`/admin/integrations`	Integration catalogue with per-integration connection probes.
`/admin/operations`	Operational tuning — SLA clocks, alert behaviour, ETA model, basemap, board windows.
`/admin/departments`	Department branding.
`/admin/automations`	NEW Automation rules, thresholds, health and run log.
`/admin/status`	NEW System status — is the platform working right now.

Operational tuning

Twelve knobs, each declaring its type, bounds, default and description in a catalogue (`packages/api/src/settings`). The admin page renders from the catalogue, so adding a knob is one entry there. Values are stored as overrides in `app_settings` , validated and clamped on both read and write, and every change is audited. If the table is unreachable the code defaults apply — a tuning problem can never take a screen down.

4.17 Criti `/ASSISTANT`

The assistant, available as a full page and as a dock on every route. It reads the live board — active incidents, queue, crews on duty, available units, today's KPIs — through the caller's own session, so RLS decides what it can see. It cannot surface a record the operator could not open themselves.

It also acts. The model emits action codes which the server validates and executes: `[UPDATE_STATUS|…]` , `[FILL_REPORT|…]` , `[ROUTE_INCIDENT|…]` , `[BROADCAST|…]` . Status values, silos and fillable fields are whitelisted server-side, and every executed action is audited. Groq is the primary provider, Gemini the fallback.

4.18 Search NEW IN WAVE 5

Finding a record used to mean knowing which screen it lived on. An operator holding a plate number, a case number or half a surname had to first decide whether that made it an incident, a vehicle, a person or a report. Under pressure that is the wrong order: you have the string, you want the record.

`search_everything()` unions six tables — incidents, personnel, fleet, police reports, fire reports, security reports — behind one query, wired into ⌘K alongside page navigation and into a shareable `/search` page.

> **TRIGRAM, NOT FULL-TEXT**
>
> Postgres's text search tokenises and stems, which is right for prose and wrong for nearly everything an operator types: `INC-20260804-0042`, `ND 123-456`, `PSV-8891`. Those are identifiers, and `to_tsvector` mangles them, so a partial identifier matches nothing. `pg_trgm` matches substrings — exactly the "I only have part of it" case — and forgives a misspelt surname. GIN trigram indexes cover every searched column.

PART V

The Responder App

> An Expo application carried by crews on shift. It is designed around three failures that will happen: no signal, a locked screen, and no free hand.

5.1 Screens

SCREEN	PURPOSE
Incidents (tab)	Assigned calls, severity-ordered. A **Mine / Nearby** switch shows every live call in the department sorted by straight-line distance — read-only, for situational awareness.
Alerts (tab)	NEW Inbox with a live unread badge and acknowledgement state.
Schedule (tab)	Current duty, next duty, upcoming allocations with call sign, vehicle and sector, leave balances, endorsements.
Shift (tab)	The three-step start-of-duty wizard and unit compliance.
Profile (tab)	Identity, duty toggle with GPS heartbeat, push diagnostics, delivery status, sign-out.
Incident detail	Full call, navigation handoff to the phone's map app, caller dial, status advance, evidence capture, timeline.
Leave	NEW Apply for leave, with balances shown before the form.
Criti	NEW The assistant, scoped to the crew member's own calls and shift.

5.2 The duty gate

A responder may not take incidents until today's readiness flow is complete. The gate is enforced in the app shell: on opening, an unfinished flow routes to the Shift tab.

Two deliberate escapes. **Profile stays reachable** — somebody who fails the readiness check can never satisfy the gate, which is correct, but they must still be able to see who they are signed in as and sign out. A gate with no exit is a trap, not a control. And if the readiness check **cannot be evaluated** because the device is offline, the crew is let through: the server remains the record of truth, and stranding a crew behind a check we cannot run helps nobody.

5.3 The offline outbox NEW IN WAVE 5

Coverage is not a solved problem. Basements, lift shafts, the industrial south, half the N3 corridor at peak. Previously a responder marking themselves On Scene in a dead zone got an error, and the update was

dropped — pushing recovery onto the person with the least attention to spare, and losing the timestamp the incident record is later reconstructed from.

Now every field mutation is written to disk first and delivered afterwards. Three consequences follow:

PROPERTY	WHY IT MATTERS
The recorded time is when the responder acted	Operations carry their own `occurredAt`, and `update_responder_status()` accepts it — bounded to the last 12 hours and never in the future, so an honest offline timestamp is possible without opening a path to backdating. An arrival time that drifts by the length of a dead zone is evidence that misleads an inquiry.
Nothing is dropped silently	An operation that exhausts its retries is parked as *failed* and stays visible in the sync tray until a human resolves it. A photograph of a scene cannot be retaken an hour later.
Replay is safe	Delivery can be interrupted after the server committed but before the phone recorded that. Every handler tolerates running twice — status changes are assignments, notes carry a zero-width replay marker, attachments derive their storage path from the operation id and upsert, and a panic is idempotent on `client_ref`.

Ordering

If the phone gets three seconds of signal, those seconds must carry the emergency, not a queue of photographs. Delivery priority is: **panic**, then status changes (dispatch allocates crews off them), then position, then notes, then attachments — the largest and least time-critical.

Collapsing

Only position heartbeats collapse: five stale fixes tell dispatch nothing the newest does not, and replaying them would walk the marker backwards across the map on reconnect. Every other operation is a distinct event with its own evidential value and is never merged.

Ownership

> **HANDSETS ARE SHARED ACROSS SHIFTS**
> Queued work is scoped to the account that captured it. Without that, an update queued by the officer going off duty would be delivered under the session of whoever signed in next — attributing their action, and their position, to the wrong person in a record later read as evidence.

Retry policy

Exponential backoff capped at 60 seconds, eight attempts. Those attempts only accrue **while the device has a connection** — the flush loop refuses to run when offline. So eight failures means the server is rejecting the write, not that the crew drove through a tunnel. Errors that will never succeed (RLS denial, missing target, 401/403/404) are not retried at all.

5.4 Officer-down SAFETY-CRITICAL

Four constraints, in the order they mattered:

1. **Usable one-handed, without looking.** One fixed target in the same place on every screen, large enough to hit blind.
2. **Cannot fire by accident.** A panic pulls crews off other calls; a few pocket presses is how a service learns to ignore the alarm. Hence hold-to-activate for 2.5 seconds, with escalating haptics so the phone confirms it is arming even face-down in a pocket.
3. **Cannot depend on the network.** The queue takes it immediately, at the highest priority.
4. **Cannot wait for GPS.** A fix can take fifteen seconds indoors. The position is raced against a four-second timeout and sent without coordinates if necessary — dispatch would rather have "officer down, position unknown" now than a precise position later.

`raise_panic()` then creates a Critical incident at the officer's position, sourced `Responder-Panic`, and raises a blocking acknowledgement-required alert to both the officer's own department and the control room. The console shows a standing banner in the app shell until the incident closes — the blocking pop-up fires once, but an operator who acknowledged it and went back to the queue needs a reminder that a colleague is still calling for help.

> **POSITION UNKNOWN**
> When no fix is available the incident is placed at the eThekwini city centre rather than at 0,0 — which is in the Gulf of Guinea and would put a marker in the ocean — and the address text says plainly that the position is unknown.

5.5 Alerting and push

Two deliberately overlapping channels.

CHANNEL	COVERS	BLIND SPOT
Supabase realtime	Everything, instantly, while the app is open — the state a crew on shift is normally in.	App closed or backgrounded.
Expo push	The phone locked in a cab holster.	Needs a native module, and FCM on Android.

The overlap is the point: either alone has a blind spot, and a missed dispatch is not a recoverable error. Duplicates are cheap by comparison and are deduplicated by alert id where the two meet. Additionally, when an alert arrives over realtime while the app is backgrounded, the device raises it as a local notification — the socket delivered it to a screen nobody is looking at.

The Android FCM contract

> **BUILD-TIME, NOT RUNTIME**
>
> Android remote push cannot work without a Firebase registration. Historically the app shipped `expo-notifications` without `google-services.json` and failed at launch — the worst outcome, because a responder with no app is worse off than a responder with no push. The dynamic Expo config now **refuses to produce an Android build** without the file, with an actionable message, unless `CRITICOM_ALLOW_NO_PUSH=1` states that a realtime-only build is wanted deliberately.

Every `expo-notifications` call is additionally guarded, so a build without the native module degrades to realtime-only rather than crashing. The Profile screen reports which case applies in plain words — permission denied, no push in this build, emulator, or registration failed — because a supervisor checking a handset before a shift needs to know whether it will ring, and whether that is fixable there or a build fact.

PART VI

The Automation Engine

> Everything Criticom did until wave 5 happened because a person did it. That is right for dispatching and wrong for noticing.

6.1 Why it exists

A call sitting unassigned for twenty minutes. A paramedic's licence lapsing next week. A duty nobody acknowledged. All three are knowable from data already in the database, and all three previously depended on somebody happening to look.

The engine lives entirely in Postgres. There is no worker process to deploy, monitor or restart, and the rules run in the same transaction boundary as the data they read.

TABLE	ROLE
`automation_rules`	One row per rule: enabled flag, interval, and a JSONB `config` holding its thresholds — so "escalate after 10 minutes" becomes 15 without a deployment.
`automation_runs`	Append-only log. Automations act on live data and raise alerts that pull crews around; when one misfires, "what did it do and when" must be answerable without reading code.
`automation_events`	The anti-spam ledger, and the most important table here.

6.2 The seven rules

RULE	EVERY	WHAT IT DOES
Unassigned call escalation sla_unassigned	5 min	A call still waiting for a department raises a High alert at the warn threshold (default 10 min) and a Critical, blocking interrupt at the critical threshold (default 20 min) — to the owning department and to the control room.
No arrival recorded sla_no_arrival	10 min	Dispatched or En Route with no arrival after 30 minutes. Confirm the crew is actually moving.
Open incident gone quiet stale_incidents	30 min	An active incident with no timeline entry for 4 hours. Bucketed by day, so a genuinely long incident — a flood, a major fire — produces one nudge a day, not one every four hours.
Certification expiry cert_expiry	12 h	Warns at 30, 7 and 1 days. To the member, because it is their licence to renew, and to the department, because rostering somebody unqualified is the department's problem first.
Unacknowledged duty duty_unacknowledged	1 h	A published duty starting within 12 hours that the member has not confirmed.
Morning digest daily_digest	30 min	Once per department per day at the configured hour: volume, criticals, closure rate, average response. Aim it at shift changeover so the incoming shift reads it.
Housekeeping hygiene · rate_limit_gc	24 h / 6 h	Prunes expired alerts, old run logs, spent idempotency records and stale rate-limit counters.

6.3 Anti-spam

> **THE CONSTRAINT THAT SHAPED THE DESIGN**
> An alerting system that cries wolf is worse than none, because people stop reading it.

Every action is claimed against `automation_events` with a unique index on `(rule_key, entity_id, bucket)`. A rule that runs every five minutes therefore alerts once per escalation tier, not every five minutes forever.

```
if public.automation_claim('sla_unassigned', inc.id::text, 'critical') then
  -- raise the alert; this branch can only ever run once for this incident/tier
end if;
```

Verified against the seeded database: the first pass raised 44 alerts across seven rules; a second pass immediately afterwards raised none.

6.4 Scheduling and failure

One `pg_cron` job, `criticom-automations`, calls `run_automations()` every five minutes. Per-rule cadence is the rule's own interval, so adding or retiming a rule is an `UPDATE`, never a schedule change — and there is exactly one job to reason about when something misfires at 03:00.

Five minutes is chosen against the tightest rule: unassigned escalation warns at ten. A tick at half the shortest threshold bounds the lateness of an escalation at five minutes, inside the tolerance of a ten-minute SLA.

One broken rule must not stop the rest

Each rule runs inside its own exception block. A rule that throws records an error against itself and the loop continues. The alternative — one broken rule silently stopping SLA escalation — is precisely the failure that must not happen quietly. Its `last_run_at` still advances, so a rule that throws every time does not retry on every tick and bury the log in identical failures.

If the scheduler is not running

The engine is complete without `pg_cron`: anything with a clock can call `run_automations()`. Migration 0026 degrades to a notice rather than failing where the extension is unavailable, so a local or self-hosted environment is not blocked by a scheduler. Admin → Automations reports a stalled scheduler explicitly, because a page full of enabled rules that have not run in a day is worse than one that says so.

PART VII

Data Model

7.1 Domains

57 tables. Grouped by what they are for:

DOMAIN	TABLES
Identity	`profiles, system_profiles, personnel, reporting_lines`
Incidents	`incidents, incident_updates, incident_attachments, incident_number_counters, citizen_share_tokens`
Fleet & devices	`apparatus, devices, cameras, hydrants`
Police	`metro_reports, metro_offenses, firearm_authorizations, security_reports, crime_stats`
Fire	`firefighter_reports, public_incident_reports`
Medical	`paramedic_reports, medical_histories, assessment_findings, vital_signs, interventions, legal_handovers, ambulance_restocks`
Disaster	`disaster_logs, post_mortem_reports, relief_stocks, mutual_aid_partners`
Shift & duty	`shifts, rosters, duty_posts, endorsements, mental_checks, vehicle_checklists`
HR	`leave_entries, training_records, certifications`
Performance	`performance_kpis, performance_plans, performance_scores`
Alerting	`alerts, alert_reads, notifications, push_tokens`
Configuration	`command_records, app_settings, integration_settings, department_branding, report_definitions`
Platform	`audit_logs, automation_rules, automation_runs, automation_events, rate_limit_buckets`

7.2 The incident record

`incidents` is the spine. Every department writes to it, and its columns encode the whole lifecycle.

GROUP	COLUMNS
Ownership	`silo, dispatched_silos[], source, incident_number, client_ref`
Caller	`caller_name, caller_number, caller_location, is_caller_patient, patient_name`
Classification	`incident_type, incident_severity, incident_category, region, fire_incident_type, medical_type, disaster_type`
Location	`location_address, suburb, city, postal_code, location_lat, location_lng, location_point (PostGIS)`
Lifecycle	`status, call_timestamp, incident_created_at, dispatch_time, arrived_timestamp, closed_at`
Assignment	`unit_id, unit_type, unit_status, responding_officer, acceptance`
Attribution	`call_taken_by, dispatched_by, created_by`

Two columns worth explaining

`dispatched_silos` is an array, and it is what makes multi-department response work: it appears in the RLS read policy, so adding a department to it grants that department visibility atomically with the routing decision.

`client_ref` is a device-generated UUID with a unique index. It exists so a replayed create — the offline outbox retrying after the network returned — returns the original row rather than raising a second emergency for the same event.

Incident numbering

> **A DEFECT FIXED IN WAVE 5**
>
> Numbers were `INC-<date>-<random 100–999>` against a `unique (silo, incident_number)` constraint. That is 900 values per department per day, so by the birthday bound a department had roughly a 50% chance of a collision at 35 calls in a day and was essentially certain past 80 — and a collision surfaced as an exception thrown at a call-taker with a caller on the line. eThekwini runs well past that. Replaced with `next_incident_number()`, a per-department daily counter producing `INC-YYYYMMDD-NNNN`.

7.3 Stored procedures

41 application functions. The ones worth knowing:

FUNCTION	SECURITY	PURPOSE
`log_incoming_call()`	Definer	Creates an incident, allocates its number, audits.
`check_duplicate_incident()`	Definer	12-hour duplicate lookup by caller and by location + type.
`route_incident()`	Definer	Routes to departments, appends timeline, audits, raises alerts — one transaction.
`dispatch_unit()`	Definer	Commits a specific appliance.
`update_responder_status()`	Definer	Status change, accepting the time the responder acted.
`raise_panic()`	Definer	Officer-down. No role gate, by design.
`close_case()`	Definer	Closure with reason, audited.
`toggle_duty()`	Definer	Duty on/off with position.
`assign_vehicle()`	Definer	Start-of-shift vehicle allocation.
`cleanup_on_logout()`	Definer	Frees the vehicle, clears duty.
`has_completed_shift_start_today()`	Definer	The duty gate.
`get_dispatcher_kpis()`	Definer	Board summary in one round trip.
`raise_dispatch_alerts()`	Invoker	Per-department alert fanout.
`count_unread_alerts()`	Invoker	Authoritative unread badge count.
`search_everything()`	Invoker	Cross-module search.
`rate_limit_hit()`	Definer	Per-user request counting.
`run_automations()`	Definer	The scheduler entry point.
`citizen_update_location()`	Definer	Public: a citizen's browser posting GPS against a single-use token.
`require_role()`	Definer	The guard opening every privileged mutator.

7.4 Migrations

28 numbered files, append-only. An applied migration is never edited — the next one is written instead. Each runs inside a transaction, so a failing statement rolls the whole file back and a half-applied migration is not a reachable state.

> **POSTGREST AND FUNCTION OVERLOADS**
> RPCs are resolved by argument name. Adding a parameter with a default to an existing function creates a *second* overload, and a call matching both resolves to neither. The old arity must be dropped in the same migration — as 0024 does for `update_responder_status` and `log_incoming_call`.

PART VIII

Integrations

8.1 Catalogue

Every external system is a row in `integration_settings`, visible with a connection probe at Admin → Integrations. Status shown is the live configuration.

CATEGORY	INTEGRATION	PURPOSE
Core	Supabase	Database, auth, storage, realtime.
	Esri World Imagery	Map basemap.
AI	Groq	Primary provider behind Criti.
	Google Gemini	Fallback provider.
Comms	RealPTT	Push-to-talk radio bridge — text to radio, audio log.
	Zoiper	Softphone dial-out.
Notifications	Expo Push	Responder app notifications.
	BulkSMS	Citizen SMS and GPS-location requests.
Surveillance	SANRAL CCTV Matrix	The current freeway camera wall.
	Hikvision HikCentral	The incoming CCTV platform.
External feeds	TomTom	Traffic incidents.
	FlightRadar24	Aircraft positions and tracks.
	AIS / Kpler	Vessel positions, Durban harbour.
	OpenWeather	Wind and weather for fire-spread modelling.
	i-Traffic	Camera registry import.

8.2 The proxy pattern

No third-party key ever reaches a client bundle. Every external call goes through a server route under `/api/external/*` which holds the key server-side, requires an authenticated session, and is rate limited.

Each proxy degrades gracefully: with no key configured it returns a clear "not configured" response rather than erroring or, worse, returning fabricated data. A control room must be able to tell "this feed is switched off" from "this feed says there is nothing there".

```
const { deny } = await requireAuth('external');   // 401 or 429
if (deny) return deny;
const key = process.env.TOMTOM_API_KEY;
if (!key) return unconfigured('TomTom traffic', 'TOMTOM_API_KEY');
```

8.3 CCTV cut-over

`/api/cctv/cameras` serves the SANRAL registry until the Hikvision integration is enabled with an app key, secret and base URL — at which point the same endpoint serves the live HikCentral estate. The cut-over is a toggle in Admin, not a deployment.

`/api/cctv/stream` brokers AK/SK-signed live-preview and recorded-playback URLs. Signing happens server-side only, and every grant is written to the audit log for chain of custody — footage retrieval is evidence handling, and who requested what has to be answerable.

PART IX

Security

9.1 Model

LAYER	CONTROL
Transport	HTTPS throughout, HSTS with a two-year max-age and preload.
Session	Supabase Auth; role and silo carried in JWT app metadata, which the client cannot alter.
Route	Middleware evaluates the route matrix and redirects to `/403`.
Row	RLS on every table. Deny by default, then explicit policies.
Function	`require_role()` opens every privileged mutator; `EXECUTE` revoked from `anon`.
Rate	Per-user limits on the routes that cost money or third-party quota.
Audit	`audit_logs` on every mutation, with actor, department and detail.

9.2 Headers and transport

Applied to every response:

HEADER	VALUE / NOTE
`Content-Security-Policy`	Self by default; Supabase REST and websockets, map tile hosts and the key-free geocoding upstreams allow-listed. `frame-ancestors 'none'`.
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload`
`X-Frame-Options`	`DENY` — defence in depth for older browsers.
`X-Content-Type-Options`	`nosniff`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	Camera, microphone and geolocation to self — the console legitimately uses all three. Everything else denied.
`Cache-Control`	`no-store` on every API response. Incident data must never sit in a shared cache.

`X-Powered-By` is disabled: version and stack disclosure serves an attacker, not an operator.

KNOWN CSP LOOSENING

`script-src` permits `'unsafe-inline'` and `'unsafe-eval'` for Next's inline bootstrap, and `style-src` permits `'unsafe-inline'` for Tailwind's runtime theme variables and per-department inline colours. Tightening this needs per-request nonces, which is a separate piece of work.

9.3 Rate limiting NEW IN WAVE 5

Several routes spend money or third-party quota per call: the assistant, geocoding, the traffic and flight and vessel proxies, PDF rendering, RealPTT, CCTV stream grants. All are authenticated already, so this is not about anonymous abuse. It is about the two things that actually happen to a running service — a bug that fires the assistant on every render and burns a month's model budget in an afternoon, and a console left unlocked in a control room, where the realistic damage is quota rather than data.

BUCKET	LIMIT	RATIONALE
`chat`	40 / min	A person types perhaps 6 a minute; a render loop types 600.
`geocode`	120 / min	Fires per keystroke-batch on the address field, so it needs headroom.
`external`	90 / min	Metered upstreams; panels poll on a timer.
`pdf`	20 / min	Embedding scene photographs is the most expensive request served.
`ptt`	60 / min	Radio traffic is bursty by nature.
`cctv`	60 / min	Each grant is audit-logged; a flood pollutes the trail.

Counted in Postgres, not in the process: the console deploys serverless, so an in-memory counter is per-instance and ten concurrent functions would mean ten times the intended limit — which is not a limit. Identity is taken from `auth.uid()` inside the database function rather than from a caller-supplied key, so a route handler cannot get it wrong and a caller cannot spend somebody else's budget or reset their own.

IT FAILS OPEN

If the limiter itself errors, the request proceeds. A rate limiter that takes down dispatch when the database hiccups has caused far more harm than the runaway loop it was guarding against.

9.4 Audit

`audit_logs` records actor, department, action type, entity, and a human-readable detail line for every mutation. Logon events are recorded there too, which is what the operational KPI logon report is computed from. Case closures, stream grants, automation changes and manual automation runs all name a person.

9.5 Outstanding items

THESE ARE OPEN AND SHOULD BE CLOSED BEFORE OR AT HANDOVER

- **V4 leaked credentials.** The previous system's BulkSMS token pair and Gemini key are present in that repository's source history — including a hardcoded token in its notifications controller. They must be **revoked at the provider**; rotating them in V5's environment is not sufficient, because the old values remain in V4's git history.
- **Setup-time credential exposure.** The Supabase service-role key and database password were pasted into a chat transcript during provisioning and should be rotated.
- **CSP nonces.** Removing `'unsafe-inline'` and `'unsafe-eval'` from `script-src`.
- **Accessibility.** 19 tracked warnings, mostly form labels not programmatically associated with their inputs on the call-taking screen — a screen reader announces those fields with no name.

PART X

Operations

> **BEFORE ANYTHING ELSE**
> Every procedure here is slower than the radio. If the platform is degraded and crews are waiting, put the control room on radio and paper, then fix the platform. Nothing in this part is more urgent than that.

10.1 Environments

CONSOLE	Vercel, built from `main`. `NEXT_PUBLIC_*` values are inlined at build time, so an environment change needs a **redeploy**, not a restart.
DATABASE	Supabase, region `eu-west-1`. The direct host is IPv6-only from some networks; CLI work goes through the session pooler.
RESPONDER APP	EAS. Channels: `development`, `preview`, `production`.

10.2 Deploying

Console

Push to `main`; CI must be green. Vercel builds the same commit.

Database

```
set -a && . ./.env && set +a
node scripts/apply-migration.mjs supabase/migrations/00NN_your_migration.sql
```

Apply migrations **before** the deployment that depends on them. The application tolerates a schema newer than the code far better than the reverse.

Responder app

```
eas build  --profile production --platform android   # native binary
eas update --branch  production --message "what changed"  # JavaScript only
```

> **AN OVER-THE-AIR UPDATE CAN ONLY EVER SHIP JAVASCRIPT**
> Adding a native module — a new Expo package, a permission, anything in `app.json` or `app.config.ts` — requires a real build. Publishing an OTA whose JavaScript imports a native module the installed binary lacks crashes every handset at launch. The defence is `runtimeVersion`: it follows the app version, so bumping the version makes older binaries ineligible for the update rather than being handed something that cannot boot. Wave 5 added five native modules and bumped the app to **1.1.0** for exactly this reason.

10.3 Backup and recovery

Supabase takes automatic daily backups; point-in-time recovery depends on the plan — **verify which you have before you need it**. Take a manual snapshot before anything irreversible:

```
pg_dump "$SUPABASE_DB_URL" --no-owner --no-acl -Fc -f criticom-$(date +%F).dump
```

Restore into a *new* project first and check it, then cut over. What a database restore does **not** bring back:

- **Storage** — `incident-media`, `avatars`, `department-logos`. Evidence photographs live here, so a restore that ignores storage silently loses the attachments a case file references.
- **Accounts created since the snapshot** — re-create with `pnpm db:create-admin`.
- **Push tokens** — harmless; phones re-register on next launch.

10.4 First response

Work in this order. Each step rules something out.

#	CHECK	INTERPRETATION
1	Open **Admin → System Status**	Each check states what it means rather than reporting a number. Start here.
2	`/api/health/config`	Reports which environment variables the running build can see, without exposing values. Missing values mean they were absent at *build* time — redeploy.
3	`psql "$SUPABASE_DB_URL" -c 'select now()'`	If this fails, everything else is downstream.
4	Sign in as a known account	Sign-in failing while the database answers points at Auth, not data.
5	Alerts in the last 15 minutes	Zero on a busy shift means realtime or the automation engine.

Common situations

SITUATION	RESPONSE
A phone is not receiving dispatches	Profile → Dispatch alerts on the handset states which case applies: permission denied (fixable there), no push in this build (needs an FCM build), emulator, or registration failed. Realtime still delivers while the app is open in all four.
"My update did not go through"	The sync tray lists everything undelivered with its error. Queued work is not lost — it is held against that person's account and sends when they next have signal while signed in.
A panic fired by accident	Close the incident as any other, with a note. The panic is deliberately easy to trigger and impossible to un-send: acknowledged and closed beats a button people hesitate over.
Alert storm from a rule	Disable the rule from Admin → Automations, then inspect `automation_events` — repeat alerting means its claim key is wrong. Widening a threshold is the fast mitigation.
Storage filling up	Scene photographs and voice notes dominate. Nothing deletes them automatically, deliberately: they are evidence. Removing any is a records-retention decision, not an operational one.

PART XI

Engineering Standards

11.1 Continuous integration

Every push and pull request. All steps block merge.

STEP	CATCHES
`pnpm lint`	Defect-class lint rules (see 11.3).
`pnpm typecheck`	Four projects: db, api, web, mobile.
`pnpm test`	47 tests across three packages.
`pnpm build`	The production console build.
Migrations from empty	All 28 migrations applied in order to a throwaway PostGIS database.

> **WHY THE MIGRATION JOB MATTERS MORE THAN IT LOOKS**
> A migration is the one artefact with no undo, and the sequence from nothing is exactly what a new environment and a restore-from-backup both execute. A migration that only works against the current production database has not been tested.

11.2 Testing

`node --test` with `--experimental-strip-types`. No framework, no transpile step, no configuration file — Node 22 runs the TypeScript directly. Tests import with explicit `.ts` extensions, which the loader requires and which Metro and `tsc` both resolve unchanged.

The tests exist where a silent regression is not a bad experience but a call that does not reach a crew:

SUITE	PINS
`roles.test.ts`	The route matrix — the authorization boundary. Including the two-desk split.
`reporting/query.test.ts`	That the report builder drops unknown columns, unsupported operators and invalid enum values, and keeps sensitive datasets closed.
`automations.test.ts`	That a zero threshold can never be stored — zero means "escalate the instant a call is logged".
`outbox-policy.test.ts`	Queue ordering, collapsing, retry classification and ownership. Every rule reachable only when the network is down.

> **A PATTERN WORTH COPYING**
>
> When safety-relevant logic is tangled with an untestable runtime (React Native, a Supabase client), split the pure decisions into their own module and test that. `outbox-policy.ts` exists purely for this; `outbox.ts` re-exports it so no call site had to change. Writing those tests immediately found a real defect — retry attempts were accruing while the device was offline, so a crew driving through a tunnel would have a scene photograph marked failed.

11.3 Linting

`pnpm lint` runs `biome lint`, not `biome check`. `check` also enforces the formatter, which disagrees with this codebase's consistent hand-formatting; running it produces a 150-file diff that changes no behaviour and buries every real finding.

Rules the house style deliberately diverges from are off, each reviewed against its actual violations and recorded with the count and reason in `docs/07-ENGINEERING-STANDARDS.md`. What remains is enforced. Accessibility rules are warnings rather than errors — real defects, tracked for repair, to be promoted back to errors once fixed.

11.4 Conventions

- Server-action forms with uncontrolled fields.
- Queries live in `packages/api/src/<module>`; pages never build queries inline.
- `logAudit` on every mutation.
- Prefer `SECURITY INVOKER` and let RLS scope; reach for `DEFINER` only where atomicity or cross-department writes demand it, and guard it with `require_role()`.
- Comments explain **why**, not what. A comment restating the code is noise; a comment recording the constraint that produced the code is the most valuable thing in the file.

PART XII

Appendices

Appendix A — API endpoints

ENDPOINT	PURPOSE
`/api/chat`	Criti. Accepts cookie or bearer authentication.
`/api/geocode`	Address ⇄ coordinate, Photon primary and Nominatim fallback.
`/api/route`	Road-routed ETA.
`/api/wind`	OpenWeather proxy.
`/api/external/traffic`	TomTom traffic incidents.
`/api/external/pois`	TomTom points of interest.
`/api/external/flights`	FlightRadar24 positions.
`/api/external/flight-tracks`	FlightRadar24 tracks.
`/api/external/vessels`	Kpler AIS vessel positions.
`/api/cameras/import`	i-Traffic camera registry import.
`/api/cctv/cameras`	Camera registry — SANRAL or HikCentral.
`/api/cctv/stream`	Signed preview and playback URLs; audit-logged.
`/api/pdf/incident/[id]`	Case-report PDF with embedded photographs.
`/api/pdf/fire-report/[id]`	Fire report PDF.
`/api/push/notify`	Expo push fanout to routed departments.
`/api/ptt/send`	RealPTT text to radio.
`/api/ptt/audio`	RealPTT two-hour audio log.
`/api/hr/report.csv`	HR report export.
`/api/performance/operational.csv`	Operational KPI export.
`/api/report-hub/export.csv`	Report hub export.
`/api/hierarchy/export.csv`	Reporting-line export.
`/api/integrations/test/[key]`	Per-integration connection probe. Admin only.
`/api/health/config`	Deployment diagnostics. Liveness public; detail admin only.

Appendix B — Migrations

#	NAME	PURPOSE
0001	`init`	Every V4 entity translated to Postgres.
0002	`rls`	Deny-by-default, then a policy per table.
0003	`realtime_storage`	Publications and storage buckets.
0004	`functions`	The multi-step workflows.
0005	`roles_alignment`	Adds the Responder and Staff roles.
0006	`seed_mutual_aid`	All 41 V4 mutual-aid partners.
0007	`v4_alignment`	Operator attribution, admin config, endorsements, performance.
0008	`fix_geo_trigger`	Fixes a nested-condition failure in the location trigger.
0009	`reporting_lines`	The reporting hierarchy.
0010	`alerts_push_integrations`	Alerts, read receipts, push tokens, integration catalogue.
0011	`route_incident`	Routing to departments without committing a unit.
0012	`app_settings_perf`	Operational tuning store and hot-path indexes.
0013	`ops_links_cctv`	Command-bar links and CCTV wiring.
0014	`hr_leave_upgrade`	Approval trail, entitlements, HR indexes.
0015	`leave_self_insert`	Members file their own leave, Pending only.
0016	`groq_integration`	Groq as primary AI provider.
0017	`department_branding`	Per-department identity.
0018	`security_hardening`	Policy tightening.
0019	`privileged_function_guards`	Role guards on every privileged RPC.
0020	`revoke_public_execute`	Makes 0019's revocations actually take effect.
0021	`reporting_duty_allocation`	Saved report definitions and duty allocation.
0022	`realtime_unread_alerts`	Live unread count in both directions.
0023	`duty_posts`	The establishment a shift must fill.
0024	`field_operations`	Offline-safe writes, honest timestamps, panic, incident numbering.
0025	`automation_engine`	Rules, run log, idempotency ledger, dispatcher.
0026	`automation_schedule`	The `pg_cron` job.
0027	`global_search`	Trigram search across six tables.

#	NAME	PURPOSE
0028	rate_limits	Per-user request counters.

Appendix C — Environment variables

VARIABLE	USED BY	EFFECT IF ABSENT
NEXT_PUBLIC_SUPABASE_URL	web	Console shows the "not configured" screen.
NEXT_PUBLIC_SUPABASE_ANON_KEY	web	As above.
SUPABASE_SERVICE_ROLE_KEY	web	Push fanout and camera import unavailable.
NEXT_PUBLIC_APP_URL	web	SMS share links fall back to the request host.
EXPO_PUBLIC_SUPABASE_URL / _ANON_KEY	mobile	App runs but cannot reach the server.
EXPO_PUBLIC_APP_URL	mobile	Criti unavailable on the phone.
GROQ_API_KEY	web	Criti falls back to Gemini.
GEMINI_API_KEY	web	With no Groq key either, Criti explains it is unconfigured.
BULKSMS_TOKEN_ID / _SECRET	web	Citizen GPS request unavailable.
REALPTT_ACCOUNT / _PASSWORD	web	PTT console reports not configured.
TOMTOM_API_KEY	web	Traffic and POI panels report not configured.
FLIGHTRADAR_API_KEY	web	Flight feeds unavailable.
MARINESIA_API_KEY	web	Vessel feed unavailable.
ITRAFFIC_API_KEY	web	Camera import unavailable.
OPENWEATHER_API_KEY	web	Wind widget unavailable.
HIKVISION_APP_KEY / _SECRET	web	CCTV stays on the SANRAL registry.
GOOGLE_SERVICES_JSON	mobile build	Android build refuses unless `CRITICOM_ALLOW_NO_PUSH=1`.
SUPABASE_DB_URL	scripts	Migration runner and seeds cannot connect.

Appendix D — Glossary

TERM	MEANING
Silo	A department, and the isolation boundary around its data.
RLS	Row-level security. Postgres deciding, per row, who may read or write it.
Route matrix	`ROUTE_RULES` — which roles may open which paths.
Two-desk split	Call-takers log; dispatchers dispatch. Deliberately separate.
Outbox	The responder app's durable queue of undelivered work.
Claim	An automation's once-only record that it has acted on a thing at a tier.
Presentation	An alert's delivery style: `toast` or blocking `modal`.
Endorsement	A duty annotation — court attendance, special event, instruction, restriction.
PIR	Post-incident review, Fire & Rescue.
PCR	Patient care record, EMS.
SIRA	Private Security Industry Regulatory Authority — governs the guard reports.
OTA	Over-the-air update. JavaScript only; never native code.
Runtime version	The native-compatibility tag matching an OTA update to binaries that can run it.

Appendix E — Further reading

DOCUMENT	CONTENTS
`docs/00-RECONNAISSANCE.md`	The V4 survey this rebuild started from.
`docs/01-ARCHITECTURE.md`	Architectural decisions in detail.
`docs/03-PHASES.md`	The migration plan.
`docs/04-SECURITY-MIGRATION.md`	Data migration and credential handling.
`docs/05-PARITY-STATUS.md`	V4 → V5 parity, wave by wave.
`docs/06-SECURITY-AUDIT.md`	The security audit and its findings.
`docs/07-ENGINEERING-STANDARDS.md`	What CI enforces and why, rule by rule.
`docs/08-RUNBOOK.md`	The on-call runbook.

Criticom V5 — Solution Wiki

Generated 4 August 2026 from the state of the repository at commit `f7909f2`. Every figure in this document — route counts, table counts, function counts, test counts — was read from the codebase and the live database rather than from memory.